

AI Engineer Roadmap - Fastest Practical Path

Your single reference document

September 2026

How To Use This Document

This is the **final, merged version** - built from two roadmaps, trimmed down to exactly what you need. Ignore everything else you've seen before this. Follow this one, in order.

Core principle: Never study a technology for more than 1-2 days without building something with it. Learn → build → break → debug → repeat.

Total time: ~10-13 weeks at 4-6 focused hours/day.

What was deliberately cut, and why: No PyTorch, no training neural networks from scratch, no deep math, no CNNs, no research-level ML. That content is for *Machine Learning Engineer / Researcher* roles, not the *AI Engineer* roles you're targeting (which build products using LLM APIs, RAG, and agents). You'll still get a working conceptual understanding of transformers below - enough to answer interview questions confidently - without the months of math that don't apply to this job.

Phase 0: Python + Git Fundamentals (Week 1-2)

Learn: - Core Python: variables, control flow, functions, classes/OOP, list/dict comprehensions, exceptions, file handling, type hints - async/await, environment variables, basic logging, pytest basics - Git + GitHub: commits, branches, pull requests

Where: - [Python Official Tutorial](#) - Corey Schafer's Python OOP/Async playlists (YouTube, free)

Assignment: CLI tool that reads a file, processes it, handles errors properly, and has at least 2 unit tests.

Phase 1: Backend Engineering - FastAPI, SQL, Docker (Week 3-4)

(This phase was missing from my original plan - it's essential and I'm folding it in.)

Learn: - REST APIs, HTTP methods, JSON, authentication basics -
FastAPI: routes, pydantic models, async endpoints, streaming responses, CORS - **PostgreSQL:** basic schema design, CRUD queries -
Docker: containerizing a Python app

Where: - [FastAPI Official Tutorial](#) - excellent, free, sufficient on its own - [PostgreSQL Tutorial \(postgresltutorial.com\)](#) - Docker's official "Get Started" guide

Project: Build a /chat API endpoint: User message → FastAPI → LLM call → response, with conversation history stored in PostgreSQL, containerized with Docker.

Rule from here on: every phase's project should run behind a FastAPI endpoint, not just a script - this builds the habit of shipping things as real services.

Phase 2: LLM APIs (Week 5)

Learn: - Anthropic & OpenAI chat completions, system prompts, streaming, temperature/top-p, structured JSON output, tool/function calling

Where: - [Anthropic API docs](#) - [Anthropic Prompt Engineering Interactive Tutorial](#) (free) - DeepLearning.AI: "ChatGPT Prompt Engineering for Developers" (free, ~1.5 hrs)

Assignments: 1. Streaming chatbot with conversation memory 2. Tool-calling script (model calls a local Python function) 3. Forced structured JSON output, validated with pydantic

Phase 3: Transformers & LLM Concepts - Conceptual Only (Week 6, ~3-4 days)

(Pulled in from the other roadmap - kept deliberately shallow. This is for interview readiness, not implementation. No PyTorch, no building one from scratch.)

Learn just enough to explain each of these in plain English: - Tokenization - how text becomes numbers - Embeddings - how meaning becomes vectors - Attention / self-attention - how a model decides what to "focus on" in context - Transformer architecture at a high level - encoder vs. decoder vs. decoder-only (what GPT/Claude are) - Context window, inference, temperature, top-p - what they control and why - Causal language modeling - why the model predicts one token at a time

Where: - [Jay Alammar's "The Illustrated Transformer"](#) (free, the best visual explanation that exists) - 3Blue1Brown's "Attention in transformers" YouTube video (free, intuitive)

Assignment: Write, in your own words, a 1-page explanation of "how does an LLM generate the next word" - good enough to say out loud in an interview. No code needed for this phase.

Phase 4: Prompt Engineering + RAG (Week 7-8)

Learn: - Few-shot prompting, chain-of-thought, prompt chaining - Embeddings, chunking strategies, vector databases (Chroma/Pinecone/Qdrant), similarity search, retrieval evaluation (groundedness, relevance)

Where: - DeepLearning.AI: *"Building Systems with the ChatGPT API"* (free) - DeepLearning.AI: *"Build LLM-Powered Apps with Vector Databases"* (free) - DeepLearning.AI: *"Building & Evaluating Advanced RAG"* (free) - [LangChain RAG tutorial](#) (free)

Project - Company/Personal Knowledge Assistant: Upload real documents (your notes, a company's docs, PDFs). Build chunk → embed → store → retrieve → answer-with-citations pipeline, served via your FastAPI backend. Then deliberately break it: ask questions with no answer in the docs, conflicting info, ambiguous phrasing - see how it fails and fix it.

Phase 5: Agents (Week 9-10)

Learn: - The agent loop: reason → act (tool call) → observe → repeat - Tool use, memory, planning, multi-step reasoning, human-in-the-loop, guardrails - MCP (Model Context Protocol) - the current standard for connecting agents to tools/data

Where: - DeepLearning.AI: *"Functions, Tools and Agents with LangChain"* (free) - DeepLearning.AI: *"AI Agents in LangGraph"* (free) - DeepLearning.AI: *"MCP: Build Rich-Context AI Apps with Anthropic"* (free) - [Hugging Face Agents Course](#) (free, thorough, has certificate)

Project - Business Automation Agent (more resume-worthy than a generic chatbot): e.g., an agent that qualifies inbound leads - extracts budget/timeline/requirements from a message, checks a CRM/database tool, drafts a follow-up, asks permission before sending, logs the activity. Multiple tools + state + failure handling.

Phase 6: Evaluation (Week 11, ~1 week)

Do not skip this - it's what separates a demo-builder from an engineer.

Learn: - Building an eval dataset: input → expected output → actual output → score → failure reason - LLM-as-judge, groundedness/faithfulness scoring, retrieval quality, latency, cost, regression testing

Where: - DeepLearning.AI: *"Production-Ready AI Agents"* (Hamel Husain, free)

Assignment: Build a small eval harness for your Phase 4 or Phase 5 project. Every time you change a prompt or pipeline, re-run it and check for regressions.

Phase 7: Ship 2-3 Portfolio Projects + Deploy (Week 12-13)

Polish your best projects and put them online - a project sitting only on your laptop doesn't count.

Deploy with: Vercel (frontend), Render or Modal (backend/agents), Streamlit Community Cloud (quick demos).

Each project needs: a clean README.md (architecture, setup, 30-second demo video/GIF), and ideally real usage (even by 5 friends).

Resume Projects - Pick 2-3, One From Each Category

1. **A deployed RAG application** solving a real problem - e.g. "Built a RAG assistant over [domain] docs, served via FastAPI, cutting document search time by X%."
2. **A multi-tool business-automation agent** - e.g. "Built an agent that qualifies leads and drafts follow-ups using tool-calling and MCP, reducing manual triage time." Shows orchestration, not just chat.
3. **An eval/observability project** - e.g. "Built an eval harness scoring LLM outputs for hallucination and relevance, catching X% of bad responses pre-production." Most self-taught engineers skip this - it's your differentiator.

Resume bullet formula: **what you built** → **what stack/technique** → **measurable outcome**.

Parallel Track: DSA (Ongoing, don't skip)

Many AI engineering interviews still include a coding round. Don't finish this roadmap and *then* start DSA - run it alongside, lighter weight.

Suggested daily split at 5 hrs/day: - Weeks 1-6: 3.5h roadmap / 1.5h DSA - Weeks 7-13: 4h roadmap / 1h DSA

Use LeetCode (free tier) - arrays, strings, hashmaps, and basic recursion cover most AI-engineering-adjacent interview rounds; you don't need to grind hard graph/DP problems unless targeting big-tech specifically.

What To Deliberately Skip For Now

PyTorch, training models from scratch, CNNs, GANs, reinforcement learning, building a transformer from scratch, CUDA/distributed training, every vector database, every agent framework, every cloud provider. Learn any of these later **only if** a specific role you're applying to actually asks for it.

The Real Target

Not: *"I know AI."*

Instead: **"Give me a business problem, and I can design, build, evaluate, deploy, and maintain a working AI system that solves it - and I can prove it with 2-3 shipped projects."**

That, plus consistent applying and interview practice once you're partway through this, is what gets you to job-ready.